



PDI: ATIVIDADE 02 · REGISTRO DE ENTREGA

Pipeline de **Testes** Automatizados (Unit/Integration/E2E) com 80% de Cobertura

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 02 · Práticas de Engenharia e Clean Code

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

pytest / gate 80% / 3 camadas

1. Contexto

Na FV, testes eram manuais e rodados só na véspera do deploy. O resultado: correção de bug gerava novo bug, e ninguém sabia se o sistema continuava funcionando após uma mudança. A confiança no código era baixa e o **medo de deploy** travava a entrega.

ANALOGIA DO CINTO DE SEGURANÇA

Dirigir sem teste automatizado é como pegar a estrada sem cinto: você até anda, mas basta um solavanco para se machucar. O teste é o cinto que prende o comportamento esperado: se algo mudar, ele avisa antes do acidente. E, como o cinto, só serve se for usado *sempre*, não só quando lembra.

2. Diagnóstico

Sem suíte automatizada, o tempo de validação cresce com o time e os retornos de bug em produção aumentam. Cada release vira uma roleta-russa. O custo de conserto sobe exponencialmente conforme o defeito chega longe.



Raiz do problema: validação sob responsabilidade humana e *opcional*. Não existe **gate objetivo** que impeça uma mudança de baixa qualidade de ir a produção.

3. Solução

Criar uma **suíte de testes automatizados em 3 camadas** com `pytest`, coberta por um **gate objetivo de 80%** aplicado no CI. As camadas: **unitário** (regra pura, rápida, sem I/O), **integração** (use case + adapter real, ex.: Postgres em container) e **e2e** (fluxo ponta a ponta via API). O threshold bloqueia o merge se a cobertura cair abaixo de 80%.

Configuração centralizada em `pyproject.toml`:

```
# pyproject.toml
[tool.pytest.ini_options]
addopts = "-q --cov=src --cov-report=term-missing "
        "--cov-fail-under=80"
testpaths = ["tests"]

[tool.coverage.run]
branch = true
source = ["src"]
```

No CI (GitHub Actions), o passo de teste falha o pipeline se a cobertura for insuficiente:

```
# .github/workflows/ci.yml
steps:
  - uses: actions/setup-python@v5
    with: { python-version: "3.12" }
  - run: pip install -e .[test]
  - run: pytest          # respeita --cov-fail-under=80
```

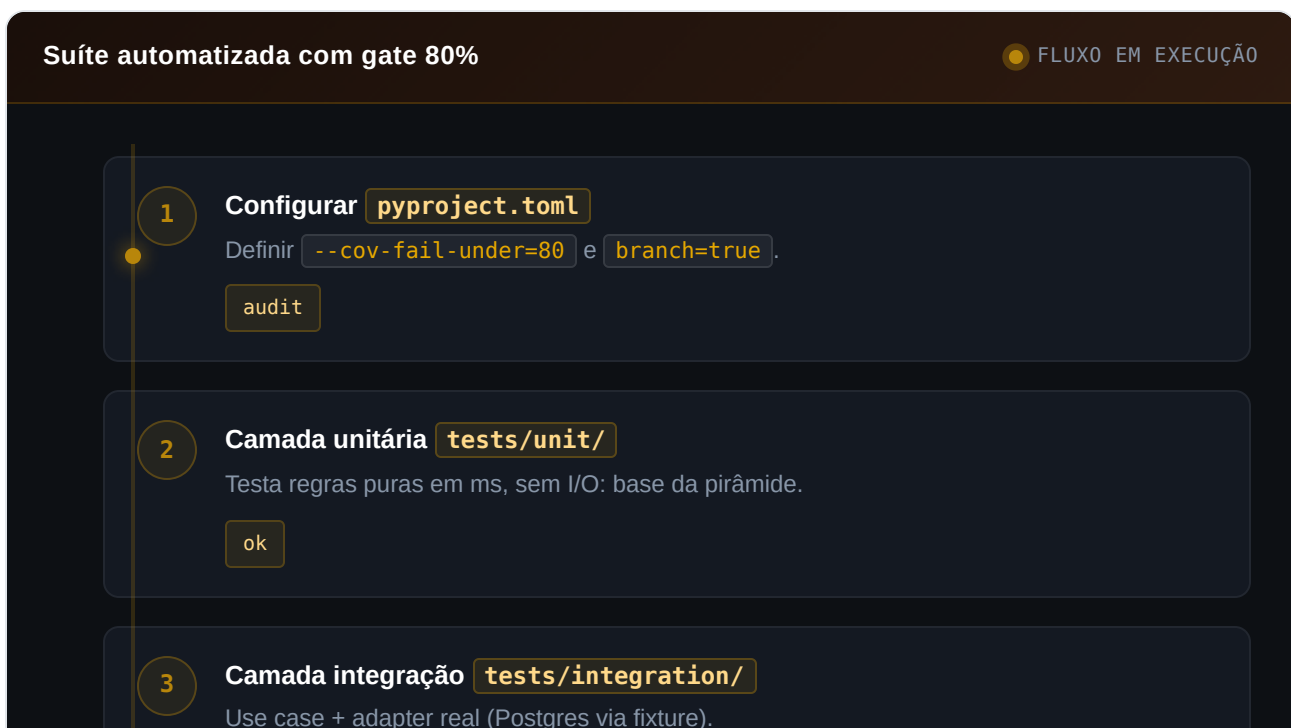
Exemplo das 3 camadas:

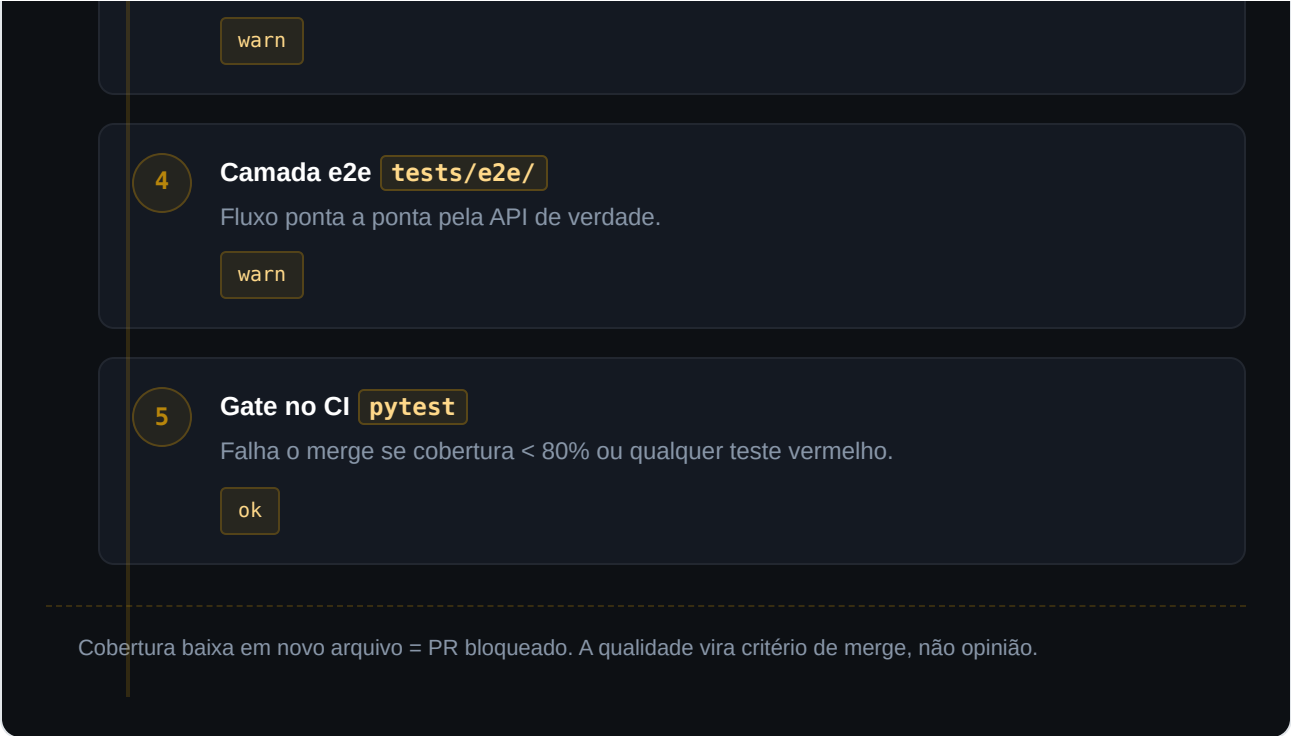
```
# unitario: regra isolada
def test_calcula_imposto_pf():
    assert CalculaImposto().executar(Pedido(100)) == 5

# integracao: use case + repo real (fixture de container)
def test_finaliza_pedido_salva(repo_postgres):
    FinalizaPedido(repo_postgres).executar(Pedido(100))
    assert repo_postgres.buscar(1) is not None

# e2e: fluxo via API
def test_post_pedido_api(client):
    r = client.post('/pedidos', json={'total': 100})
    assert r.status_code == 201
```

4. Como funciona (pipeline)





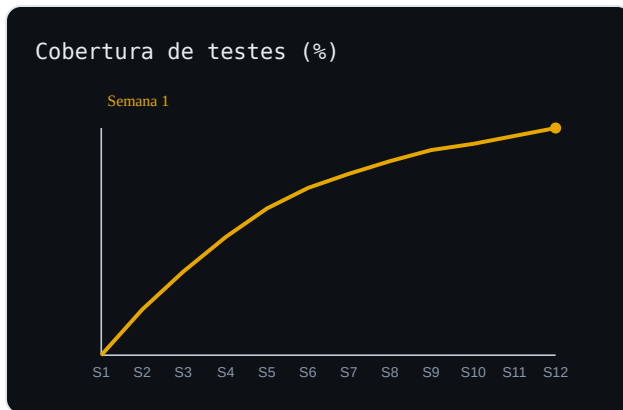
5. Antes vs Depois

CENÁRIO	ANTES (MANUAL)	DEPOIS (GATE 80%)
Validação de release	Dias de QA manual	Minutos no CI
Defeito em produção	Comum	Raro (e2e pega)
Cobertura conhecida	Desconhecida	Medida e bloqueante
Confiança no deploy	Baixa	Alta

6. Entregas

- **Config** `pyproject.toml`: `addopts` com `--cov-fail-under=80` e `branch`.
- **Pastas** `tests/unit`, `tests/integration`, `tests/e2e` com exemplos reais.
- **Workflow** `ci.yml`: passo de `pytest` como gate de merge.
- **Doc** `TESTING.md`: como rodar cada camada localmente.

7. Métricas



Meta: sustentar $\geq 80\%$ de cobertura com branch coverage e zero teste vermelho no main.

9. Decisoes e tradeoffs

- pytest com testcontainers e playwright escolhido sobre so unitarios mockados: cobre 3 camadas com realismo e o custo maior de setup compensa, pois unitario sozinho e cego a integracao.
- Unitario mira logica pura com alvo de 90 por cento, integracao mira ports com DB efemero com alvo de 80 por cento, e E2E cobre so happy path: camadas rapidas seguram o merge e a camada lenta nao trava o time.
- Gate de cobertura minima de 80 por cento com `--cov-fail-under=80` como required check no CI: impede piorar a cobertura sem perceber, saindo de 0 por cento e CI que so rodava lint.
- E2E em stage separado com retry, fora do caminho critico do merge: evita que teste lento ou instavel bloqueie o fluxo diario dos cerca de 30 modulos Python e nos JS.
- Fixtures isoladas com retry de 1 vez e isolamento contra estado global: sustenta tempo de unit e integracao menor que 3 min e flaky rate menor que 1 por cento.

10. Impacto no negocio

O projeto tem cerca de 30 modulos Python mais nos JS e o CI atual so roda lint, entao refactor de prompt ou tool injeta regressao em producao sem rede de segurancia. O gate de 80 por cento com unit e integracao em menos de 3 min troca dias de validacao manual por minutos no CI, reduz regressao frequente para rara com flaky abaixo de 1 por cento, e evita o custo de corrigir defeito tarde, quando ele ja chegou a main e a producao.

11. Referencias de estudo

- Curso: Testes automatizados com pytest, na Alura.

- Video: Piramide de testes na pratica com Python, no YouTube.
- Doc oficial: Documentacao do pytest sobre execucao e cobertura, em docs.pytest.org.
- Doc oficial: Documentacao do Coverage.py sobre medicao com branch, em coverage.readthedocs.io.

Nota de autoria: secoes de retrofit derivadas do conteudo ja existente na atividade, sem metricas novas e sem links nao verificados.

8. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.